

dbt Workflow Guide & CLI Cheat Sheet

A practical reference for using dbt Core with Databricks across any project.

How to Use dbt in Any New Project

The Pattern (Same Every Time)

Every dbt project follows the same structure regardless of what you're building:

```
your_project/
├─ venv/                # Python virtual environment
├─ .env                 # secrets – never commit
├─ dbt_your_project/
│   ├─ dbt_project.yml  # project config
│   ├─ packages.yml     # extra packages
│   ├─ models/
│   │   ├─ sources.yml  # define your source tables
│   │   └─ gold/        # your transformation SQL files
│   └─ schema.yml       # data quality tests
```

And `~/.dbt/profiles.yml` holds your connection details (shared across all projects).

Step-by-Step: Starting a New dbt Project

1. Create your virtual environment and install dbt

```
powershell

python -m venv venv
venv\Scripts\activate
pip install dbt-core dbt-databricks
```

2. Initialise the project

```
powershell

dbt init your_project_name
```

When prompted: choose databricks, enter your host, http_path, token, catalog, schema, threads.

3. Verify your profiles.yml was created

```
powershell  
  
notepad $env:USERPROFILE\.dbt\profiles.yml
```

It should look like:

```
yaml  
  
your_project_name:  
  target: dev  
  outputs:  
    dev:  
      type: databricks  
      host: your-workspace.cloud.databricks.com  
      http_path: /sql/1.0/warehouses/your-warehouse-id  
      token: "{{ env_var('YOUR_TOKEN') }}"  
      catalog: your_catalog  
      schema: your_schema  
      threads: 4
```

4. Set your token as an environment variable

```
powershell  
  
$env:YOUR_TOKEN = "your-databricks-token"
```

5. Test the connection

```
powershell  
  
cd your_project_name  
dbt debug  
# Should end with: All checks passed!
```

6. Define your sources

Create `models/sources.yml` to tell dbt where your input tables are:

```
yaml
```

```
version: 2
sources:
  - name: silver
    catalog: your_catalog
    schema: silver
    tables:
      - name: your_table_name
```

7. Write your transformation models

Each `.sql` file in `models/` becomes a table in Databricks. Always start with a config block:

```
sql

{{ config(
  materialized = 'table',
  file_format  = 'delta'
) }}

select * from {{ source('silver', 'your_table') }}
```

8. Add data quality tests in schema.yml

```
yaml

version: 2
models:
  - name: your_model
    columns:
      - name: id
        tests: [not_null, unique]
```

9. Build and test

```
powershell

dbt build
```

Profiles.yml — Multiple Projects

Your `~/dbt/profiles.yml` can hold connections for multiple projects at once:

yaml

```
project_one:
  target: dev
  outputs:
    dev:
      type: databricks
      host: workspace-one.cloud.databricks.com
      http_path: /sql/1.0/warehouses/abc123
      token: "{{ env_var('DATABRICKS_TOKEN') }}"
      catalog: project_one
      schema: gold
      threads: 4

project_two:
  target: dev
  outputs:
    dev:
      type: databricks
      host: workspace-two.cloud.databricks.com
      http_path: /sql/1.0/warehouses/def456
      token: "{{ env_var('DATABRICKS_TOKEN_TWO') }}"
      catalog: project_two
      schema: reporting
      threads: 4
```

Each `dbt_project.yml` references its profile by name:

yaml

```
profile: project_one
```

CLI Cheat Sheet

Daily Commands

Command	What it does
<code>dbt build</code>	Build all models + run all tests (use this daily)
<code>dbt run</code>	Build all models only

Command	What it does
<code>dbt test</code>	Run all tests only
<code>dbt debug</code>	Test your connection to Databricks
<code>dbt deps</code>	Install packages from packages.yml

Targeting Specific Models

Command	What it does
<code>dbt run --select model_name</code>	Build one specific model
<code>dbt run --select folder.*</code>	Build all models in a folder
<code>dbt run --select +model_name</code>	Build a model and all its upstream dependencies
<code>dbt run --select model_name+</code>	Build a model and all its downstream dependents
<code>dbt test --select model_name</code>	Test one specific model
<code>dbt build --select model_name</code>	Build + test one specific model

Refresh and Clean

Command	What it does
<code>dbt run --full-refresh</code>	Drop and recreate all tables from scratch
<code>dbt run --select model_name --full-refresh</code>	Full refresh one model only
<code>dbt clean</code>	Delete the target/ and dbt_packages/ folders
<code>dbt deps</code>	Re-install packages after dbt clean

Documentation

Command	What it does
<code>dbt docs generate</code>	Build the documentation site
<code>dbt docs serve</code>	Open docs in browser at localhost:8080

Useful Flags

Flag	What it does
<code>--target prod</code>	Use the prod connection instead of dev
<code>--vars '{"key": "value"}'</code>	Pass a variable into your models
<code>--no-partial-parse</code>	Force a full re-parse (fixes some cache issues)
<code>--debug</code>	Show verbose output for troubleshooting

Materialization Types

When you write `{{ config(materialized = '...') }}` at the top of a model, you choose how dbt stores it:

Type	What it does	When to use
<code>table</code>	Drops and recreates the full table every run	Gold/reporting layers
<code>view</code>	Creates a SQL view (no data stored)	Simple transformations, dev work
<code>incremental</code>	Only processes new/changed rows	Large tables where full refresh is slow
<code>ephemeral</code>	Temporary CTE — not stored in database	Intermediate logic you don't need to query directly

Source vs Ref

Two key dbt functions you'll use constantly:

`{{ source('schema_name', 'table_name') }}` — reference a table that exists outside dbt (your Silver tables, raw data, etc.)

sql

```
select * from {{ source('silver', 'transaction_headers') }}
```

`{{ ref('model_name') }}` — reference another dbt model (one you built yourself)

```
sql  
  
select * from {{ ref('gold_sales_daily') }}
```

Always use these instead of hardcoding table names — dbt uses them to build the lineage graph and resolve dependencies automatically.

Keeping Your Token Set

Every time you open a new terminal you need to set the token again:

```
powershell  
  
$env:DATABRICKS_TOKEN = "your-token-here"
```

To avoid typing this every session, add it to your PowerShell profile:

```
powershell  
  
notepad $PROFILE
```

Add this line and save:

```
powershell  
  
$env:DATABRICKS_TOKEN = "your-token-here"
```

Now it sets automatically every time you open a terminal.

dbt Core + Databricks — Workflow Reference Stack: Python 3.11 + dbt-core + dbt-databricks + Databricks Free Edition